



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Inurreta Brito Gustavo Enrique

Introducción

En esta actividad analicé el caso del sistema GymManager para identificar los problemas que afectan la calidad del software. También revisé cómo aplicar los principios de Clean Code y SOLID, la deuda técnica, el MVP y la Definition of Done. Todo esto me ayudó a entender la importancia de desarrollar un software bien organizado y fácil de mantener.

Parte 1.- Identificación del problema

Lee cuidadosamente el caso de estudio.

Identifica al menos 12 problemas presentes en el proyecto.

Para cada problema deberás indicar:

- ¿Qué ocurre?
- ¿Por qué representa una mala práctica?
- ¿Qué consecuencias podría generar durante el desarrollo o mantenimiento del sistema?

1. Clases con más de 1,800 líneas de código

¿Qué ocurre?

Hay clases que tienen más de 1,800 líneas de código, por lo que concentran demasiadas cosas en un solo lugar.

¿Por qué representa una mala práctica?

Porque cuando una clase es tan grande resulta más difícil entender qué hace y encontrar la parte que se necesita modificar.

¿Qué consecuencias podría generar?

Puede hacer que el mantenimiento sea más lento, que aparezcan más errores y que agregar nuevas funciones sea más complicado.

2. Funciones con más de 250 líneas

¿Qué ocurre?

Algunas funciones tienen más de 250 líneas de código.

¿Por qué representa una mala práctica?

Porque una función debería hacer una sola tarea. Si tiene demasiadas líneas, es más difícil entender cómo funciona.

¿Qué consecuencias podría generar?

Será más complicado corregir errores, hacer cambios y encontrar la parte del código que está fallando.

3. Se copia y pega código entre módulos

¿Qué ocurre?

En lugar de reutilizar el código correctamente, los desarrolladores lo copian y lo pegan en diferentes partes del sistema.

¿Por qué representa una mala práctica?

Porque si ese código tiene un error o necesita una mejora, habrá que cambiarlo en todos los lugares donde fue copiado.

¿Qué consecuencias podría generar?

Se pierde tiempo haciendo cambios y es más fácil que unas partes del sistema queden diferentes de otras.

4. Variables con nombres poco claros

¿Qué ocurre?

Se usan nombres como x, dato, temp, valor1 u obj, que no explican para qué sirven.

¿Por qué representa una mala práctica?

Porque cuando otra persona lea el código no entenderá fácilmente qué significa cada variable.

¿Qué consecuencias podría generar?

El código será más difícil de comprender y aumentará la posibilidad de cometer errores al modificarlo.

5. Un solo archivo hace muchas cosas

¿Qué ocurre?

Un mismo archivo valida datos, consulta la base de datos, genera reportes y también envía correos.

¿Por qué representa una mala práctica?

Porque reúne demasiadas tareas en un solo archivo, haciendo que todo sea más desordenado.

¿Qué consecuencias podría generar?

Si se modifica una parte del archivo, se podrían afectar otras funciones sin darse cuenta.

6. Cada desarrollador organiza el proyecto de forma diferente

¿Qué ocurre?

Cada integrante del equipo acomoda los archivos de una manera distinta.

¿Por qué representa una mala práctica?

Porque no existe un orden que todos sigan y eso hace más difícil trabajar en equipo.

¿Qué consecuencias podría generar?

Los desarrolladores tardarán más en encontrar el código y el proyecto será más difícil de mantener.

7. No existe una guía para programar

¿Qué ocurre?

El proyecto no tiene reglas o una guía para escribir el código.

¿Por qué representa una mala práctica?

Porque cada desarrollador programa a su manera y el código termina siendo diferente en cada parte del sistema.

¿Qué consecuencias podría generar?

El proyecto se vuelve más desordenado y el mantenimiento toma más tiempo.

8. Se usan demasiados comentarios

¿Qué ocurre?

Los comentarios explican prácticamente cada línea del código.

¿Por qué representa una mala práctica?

Porque lo ideal es que el código sea claro por sí mismo y no dependa de tantos comentarios para entenderse.

¿Qué consecuencias podría generar?

Si el código cambia y los comentarios no se actualizan, pueden causar confusión.

9. Las funciones se dan por terminadas solo porque parecen funcionar

¿Qué ocurre?

Las funcionalidades se consideran listas únicamente porque aparentan funcionar.

¿Por qué representa una mala práctica?

Porque no se comprueba realmente si todo funciona correctamente antes de entregarlo.

¿Qué consecuencias podría generar?

Es posible que lleguen errores al sistema y que los usuarios tengan problemas al utilizarlo.

10. No revisan el código entre compañeros

¿Qué ocurre?

Los desarrolladores no revisan el trabajo de los demás antes de integrarlo al proyecto.

¿Por qué representa una mala práctica?

Porque una revisión ayuda a encontrar errores y mejorar la calidad del código.

¿Qué consecuencias podría generar?

Se pueden pasar errores que después serán más difíciles y costosos de corregir.

11. Agregan funciones que el cliente nunca pidió

¿Qué ocurre?

El equipo desarrolla funciones que cree que pueden ser útiles, aunque el cliente no las haya solicitado.

¿Por qué representa una mala práctica?

Porque se invierte tiempo en cosas que no eran necesarias para el proyecto.

¿Qué consecuencias podría generar?

El desarrollo puede retrasarse y aumentar los costos sin aportar un beneficio real para el cliente.

12. La documentación nunca se actualiza

¿Qué ocurre?

La documentación del proyecto se queda igual aunque el sistema siga cambiando.

¿Por qué representa una mala práctica?

Porque la información deja de coincidir con el funcionamiento real del sistema.

¿Qué consecuencias podría generar?

Será más difícil hacer mantenimiento y los nuevos desarrolladores tardarán más en entender cómo funciona el proyecto.

Parte 2.- Análisis desde Clean Code

Selecciona cinco problemas relacionados con la calidad del código.

Para cada uno explica:

- Qué principio de Clean Code se está incumpliendo.
- Cómo debería corregirse.
- Qué beneficios tendría aplicar esa mejora.

Problema 1. Clases con más de 1,800 líneas

¿Qué principio de Clean Code se está incumpliendo?

Se está incumpliendo el principio de que una clase no debe ser demasiado grande y debe encargarse de una sola función. Si una clase hace muchas cosas, es más difícil entender cómo funciona.

¿Cómo debería corregirse?

Lo mejor sería dividir esa clase en varias clases más pequeñas, donde cada una tenga una tarea específica.

¿Qué beneficios tendría aplicar esa mejora?

El código sería más fácil de entender, de modificar y de mantener. Además, encontrar errores sería mucho más sencillo.

Problema 2. Funciones con más de 250 líneas

¿Qué principio de Clean Code se está incumpliendo?

Se incumple el principio de crear funciones pequeñas y fáciles de leer. Una función muy larga normalmente hace varias tareas al mismo tiempo.

¿Cómo debería corregirse?

Se debería dividir la función en varias funciones más pequeñas para que cada una haga una sola tarea.

¿Qué beneficios tendría aplicar esa mejora?

El código sería más ordenado, sería más fácil encontrar errores y cualquier cambio se podría hacer de una manera más rápida.

Problema 3. Se copia y pega código entre módulos

¿Qué principio de Clean Code se está incumpliendo?

Se está incumpliendo el principio de evitar repetir código. Copiar y pegar la misma información en varias partes hace que el sistema sea más difícil de mantener.

¿Cómo debería corregirse?

Se debería crear una sola parte del código que pueda ser utilizada por todos los módulos sin necesidad de copiarla.

¿Qué beneficios tendría aplicar esa mejora?

Se ahorraría tiempo al hacer cambios, habría menos errores y el código sería más fácil de mantener.

Problema 4. Variables con nombres poco claros

¿Qué principio de Clean Code se está incumpliendo?

Se incumple el principio de utilizar nombres claros y fáciles de entender. Los nombres de las variables deben indicar para qué sirven.

¿Cómo debería corregirse?

Se deberían cambiar los nombres por otros que describan mejor la información que almacenan.

¿Qué beneficios tendría aplicar esa mejora?

El código sería mucho más fácil de leer y cualquier desarrollador podría entenderlo rápidamente sin confundirse.

Problema 5. Un mismo archivo hace muchas tareas

¿Qué principio de Clean Code se está incumpliendo?

Se incumple el principio de mantener el código organizado. Un solo archivo no debería encargarse de tantas tareas diferentes.

¿Cómo debería corregirse?

Se debería separar cada tarea en archivos diferentes, para que cada uno tenga una responsabilidad específica.

¿Qué beneficios tendría aplicar esa mejora?

El proyecto sería más ordenado, sería más fácil hacer cambios y habría menos posibilidades de afectar otras partes del sistema al modificar el código.

Parte 3.- Aplicación de los principios SOLID

Analiza el caso e identifica qué principios SOLID no están siendo respetados.

Para cada principio encontrado explica:

- En qué consiste.
- Qué parte del caso evidencia que no se está aplicando.
- Qué cambios realizarías para cumplir dicho principio.
- Qué beneficios tendría para el sistema.

No es obligatorio encontrar los cinco principios; identifica únicamente aquellos que realmente apliquen al caso y justifica tu respuesta.

Principio de Responsabilidad Única

¿En qué consiste?

Este principio dice que una clase o un archivo debe encargarse de una sola tarea y no de varias al mismo tiempo.

¿Qué parte del caso demuestra que no se está aplicando?

En el caso se menciona que un mismo archivo valida datos, consulta la base de datos, genera reportes y además envía correos electrónicos. Eso significa que hace demasiadas cosas.

¿Qué cambios realizarías para cumplir este principio?

Separaría esas tareas en diferentes archivos o clases, para que cada uno tenga una función específica.

¿Qué beneficios tendría para el sistema?

El sistema sería más ordenado, sería más fácil encontrar errores y hacer cambios sin afectar otras partes del proyecto.

Principio Abierto/Cerrado

¿En qué consiste?

Este principio dice que el sistema debe permitir agregar nuevas funciones sin tener que modificar el código que ya funciona.

¿Qué parte del caso demuestra que no se está aplicando?

En el caso se dice que cada nueva funcionalidad requiere modificar varias clases existentes.

¿Qué cambios realizarías para cumplir este principio?

Organizaría mejor el código para que las nuevas funciones se puedan agregar sin tener que cambiar tantas partes del sistema.

¿Qué beneficios tendría para el sistema?

Sería más fácil agregar nuevas funciones, habría menos errores y el mantenimiento sería más rápido.

Principio de Segregación de Interfaces

¿En qué consiste?

Este principio dice que cada parte del sistema debe tener solo las funciones que realmente necesita y no depender de cosas que no utiliza.

¿Qué parte del caso demuestra que no se está aplicando?

Al tener archivos que realizan muchas tareas diferentes, es señal de que el sistema no está bien dividido y algunas partes dependen de funciones que no les corresponden.

¿Qué cambios realizarías para cumplir este principio?

Dividiría el sistema en módulos más pequeños, donde cada uno tenga únicamente las funciones que necesita.

¿Qué beneficios tendría para el sistema?

El código sería más organizado, más fácil de entender y mucho más sencillo de mantener.

Principio de Inversión de Dependencias

¿En qué consiste?

Este principio dice que el código debe depender de elementos generales y no estar unido directamente a una sola implementación.

¿Qué parte del caso demuestra que no se está aplicando?

Como cada nueva función obliga a modificar varias clases, significa que muchas partes del sistema dependen unas de otras de forma directa.

¿Qué cambios realizarías para cumplir este principio?

Haría que cada módulo trabajara de forma más independiente para evitar que un cambio afecte a todo el sistema.

¿Qué beneficios tendría para el sistema?

El proyecto sería más flexible, sería más fácil hacer cambios y habría menos problemas al agregar nuevas funcionalidades.

Parte 4.- Deuda Técnica

Identifica las decisiones que generan deuda técnica.

Posteriormente responde:

- ¿Qué tipo de deuda técnica observas en el proyecto?

Se copia y pega código entre módulos, existen clases y funciones demasiado grandes, no se hacen revisiones de código, la documentación no se actualiza y también se dejan errores sin corregir para avanzar más rápido

- ¿Cómo afectará al mantenimiento del sistema?

La deuda técnica hará que el mantenimiento sea más complicado porque cada vez que se quiera hacer un cambio habrá que revisar muchas partes del código.

También será más difícil encontrar errores y entender cómo funciona el sistema, especialmente para los nuevos desarrolladores.

- ¿Qué impacto tendrá sobre el tiempo y costo del desarrollo?

El desarrollo será más lento porque se invertirá más tiempo corrigiendo errores que creando nuevas funciones.

Además, el costo del proyecto aumentará, ya que será necesario dedicar más horas de trabajo al mantenimiento y a solucionar problemas que pudieron evitarse desde el principio.

- ¿Qué acciones propondrías para disminuir dicha deuda?

Yo propondría organizar mejor el código, eliminar el código repetido, dividir las clases y funciones que son demasiado grandes y mantener la documentación actualizada.

Parte 5.- Producto Mínimo Viable (MVP)

Durante el desarrollo, el cliente solicita agregar las siguientes funcionalidades:

- Chat interno entre entrenadores.
- Integración con relojes inteligentes.
- Sistema de recompensas.
- Publicación automática en redes sociales.
- Tienda en línea de suplementos.
- Gestión de clientes.
- Registro de pagos.
- Control de asistencia.
- Reservación de clases.
- Reportes básicos de ingresos.

Como líder del proyecto, determina:

- ¿Cuáles funcionalidades formarían parte del MVP?

1. Gestión de clientes
2. Registro de pagos
3. Control de asistencia
4. Reservación de clases
5. Reportes básicos de ingresos

- ¿Cuáles dejarías para futuras versiones?

1. Chat interno entre entrenadores
2. Integración con relojes inteligentes
3. Sistema de recompensas
4. Publicación automática en redes sociales
5. Tienda en línea de suplementos

- Justifica cada decisión considerando el valor para el cliente, el tiempo de desarrollo y el impacto en el negocio.

Justificación de Funcionalidades básicas del MVP

Gestión de clientes

Justificación:

La incluiría porque es una de las funciones principales del sistema. Sin el registro de clientes, el gimnasio no podría administrar a sus usuarios ni llevar un control de la información.

Registro de pagos

Justificación:

También la incluiría porque permite llevar el control de los pagos de las membresías. Es una función importante para el funcionamiento del negocio y para conocer los ingresos del gimnasio.

Control de asistencia

Justificación:

La agregaría porque ayuda a registrar la entrada de los clientes y permite llevar un mejor control del uso del gimnasio desde el primer día.

Reservación de clases

Justificación:

La incluiría porque facilita que los clientes puedan apartar un lugar en las clases disponibles y ayuda a organizar mejor los horarios.

Reportes básicos de ingresos

Justificación:

Los reportes básicos son importantes para que el administrador pueda revisar cuánto dinero está generando el gimnasio sin necesidad de desarrollar reportes más complejos.

Justificación de Funcionalidades que dejaría para futuras versiones

Chat interno entre entrenadores

Justificación:

Lo dejaría para después porque el gimnasio puede funcionar sin esta herramienta. Aunque puede ser útil, no es indispensable para iniciar el sistema.

Integración con relojes inteligentes

Justificación:

La dejaría para una versión futura porque requiere más tiempo de desarrollo y el gimnasio puede trabajar normalmente sin esta integración.

Sistema de recompensas

Justificación:

Lo agregaría más adelante porque primero es más importante que el sistema funcione correctamente con las funciones principales. Después se pueden añadir herramientas para mejorar la experiencia de los clientes.

Publicación automática en redes sociales

Justificación:

No la considero una prioridad, ya que no afecta el funcionamiento del gimnasio. Es una función que puede desarrollarse cuando el sistema principal ya esté terminado.

Tienda en línea de suplementos

Justificación:

También la dejaría para futuras versiones porque representa un módulo completamente nuevo. Lo más importante al inicio es administrar el gimnasio y no vender productos.

Parte 6.- Definition of Done (DoD)

Diseña una Definition of Done para este proyecto.

Deberá contener al menos 10 criterios que una funcionalidad debe cumplir antes de considerarse terminada.

Los criterios deberán abarcar aspectos como:

- Calidad del código.
- Funcionamiento.
- Pruebas.
- Documentación.
- Integración.
- Revisión de código.
- Cumplimiento de requerimientos.

Cada criterio deberá estar redactado de forma clara y verificable.

1. La funcionalidad cumple con todos los requisitos que pidió el cliente.
2. El código funciona correctamente y no presenta errores durante las pruebas.
3. El código está organizado y es fácil de entender.
4. Se realizaron pruebas para comprobar que la funcionalidad trabaja correctamente.
5. La nueva funcionalidad no afecta el funcionamiento de las demás partes del sistema.
6. Otro integrante del equipo revisó el código antes de integrarlo al proyecto.
7. La documentación del proyecto fue actualizada con los cambios realizados.
8. Los nombres de las variables, funciones y clases son claros y fáciles de comprender.
9. La funcionalidad fue integrada correctamente al proyecto principal y funciona sin problemas.
10. El cliente o el responsable del proyecto revisó la funcionalidad y confirmó que cumple con lo solicitado.

Parte 7.- Reflexión Final

Redacta una conclusión donde respondas:

- ¿Cuál de los temas vistos en esta unidad consideras más importante para desarrollar software de calidad?

Clean Code, porque me hizo entender que no basta con que un programa funcione, sino que también debe estar bien organizado y ser fácil de entender para las personas que le den mantenimiento a la página después de entregarla

- **¿Qué consecuencias tendría ignorar estos principios en un proyecto real?**

el proyecto se vuelve más difícil de mantener. Cada cambio puede generar nuevos errores, el desarrollo se hace más lento y los costos aumentan

- **¿Cómo aplicarás estos conocimientos en tu propio proyecto de la asignatura?**

Organizando el código, utilizando nombres claros para las variables y funciones procurando que cada clase o función tenga una sola responsabilidad.

aparte con testeos y pruebas de la página para afinar posibles errores antes de la entrega

Conclusión

Con esta actividad aprendí que no basta con que un programa funcione, también es importante que el código esté bien organizado y sea fácil de entender. Conocer los principios de Clean Code y SOLID me hizo darme cuenta de que ayudan a evitar muchos problemas durante el desarrollo. En mi proyecto voy a tratar de aplicar estas buenas prácticas para escribir un código más claro, evitar errores y hacer que el sistema sea más fácil de mantener.